

Varsha M

Data Analysis Track

Task 4

```
import pandas as pd
import numpy as np
import re
import string
import matplotlib.pyplot as plt
import seaborn as sns
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report)
from wordcloud import WordCloud
```

```
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
df = pd.read_csv("Tweets.csv")
df.head()
```

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment
0	570306133677760513	neutral	1.0000	NaN	NaN	Virgin America	
1	570301130888122368	positive	0.3486	NaN	0.0000	Virgin America	
2	570301083672813571	neutral	0.6837	NaN	NaN	Virgin America	
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Virgin America	
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Virgin America	

```
df = df[['airline_sentiment', 'text']]
df.head()
```

	airline_sentiment	text
0	neutral	@VirginAmerica What @dhepburn said.
1	positive	@VirginAmerica plus you've added commercials t...
2	neutral	@VirginAmerica I didn't today... Must mean I n...
3	negative	@VirginAmerica it's really aggressive to blast...
4	negative	@VirginAmerica and it's a really big bad thing...

```
print("Shape:", df.shape)
print(df.isnull().sum())
```

```
Shape: (14640, 2)
airline_sentiment    0
text                 0
dtype: int64
```

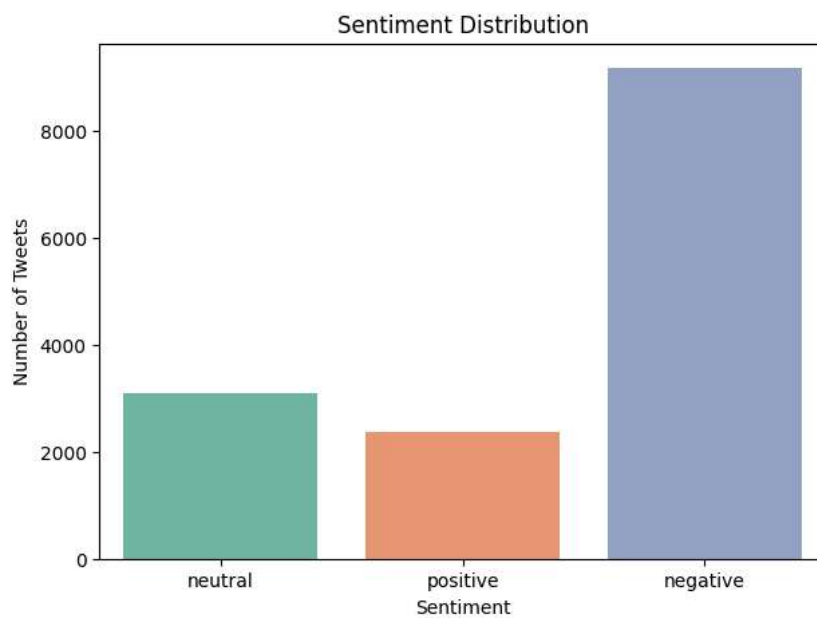
```
df = df.dropna()
print("Shape after cleaning:", df.shape)
```

```
Shape after cleaning: (14640, 2)
```

```
print(df['airline_sentiment'].value_counts())
```

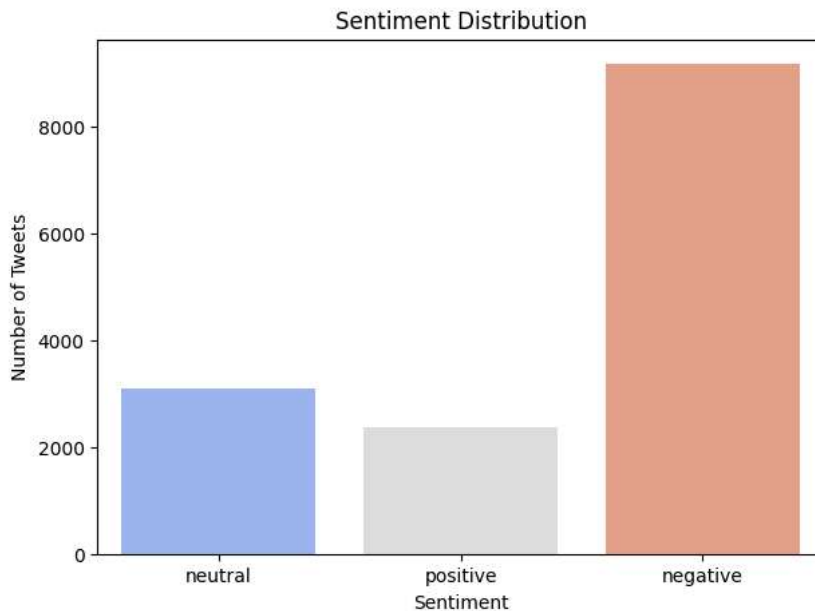
```
airline_sentiment
negative    9178
neutral    3099
positive    2363
Name: count, dtype: int64
```

```
plt.figure(figsize=(7,5))
sns.countplot(
    data=df,
    x='airline_sentiment',
    hue='airline_sentiment',
    palette='Set2',
    legend=False)
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Tweets")
plt.show()
```



```
plt.figure(figsize=(7,5))
sns.countplot(
    data=df,
    x='airline_sentiment',
    hue='airline_sentiment',
```

```
palette='coolwarm',
legend=False)
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Tweets")
plt.show()
```



```
stop_words = set(stopwords.words('english'))
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(tokens)
df['clean_text'] = df['text'].apply(clean_text)
df[['text', 'clean_text']].head()
```

	text	clean_text
0	@VirginAmerica What @dhepburn said.	said
1	@VirginAmerica plus you've added commercials t...	plus youve added commercials experience tacky
2	@VirginAmerica I didn't today... Must mean I n...	didnt today must mean need take another trip
3	@VirginAmerica it's really aggressive to blast...	really aggressive blast obnoxious entertainmen...
4	@VirginAmerica and it's a really big bad thing...	really big bad thing

```
tfidf = TfidfVectorizer(
    max_features=5000)
X = tfidf.fit_transform(df['clean_text'])
y = df['airline_sentiment']
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42,stratify=y)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (11712, 5000)
Testing data: (2928, 5000)
```

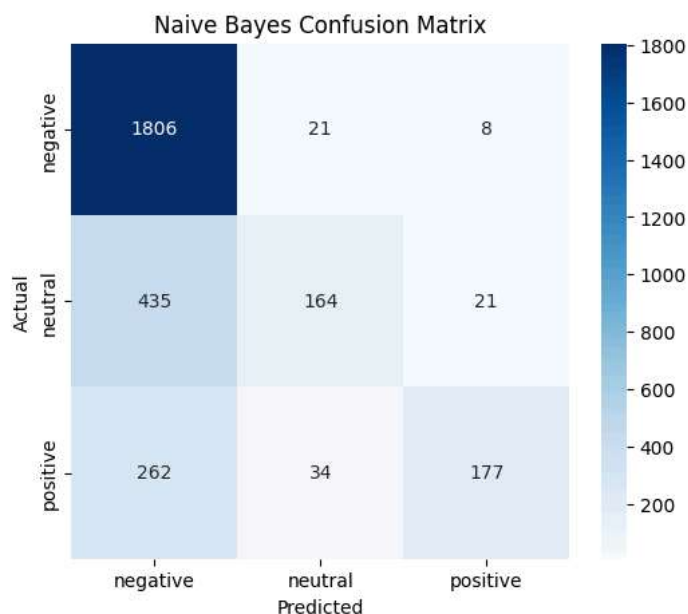
```
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
nb_pred = nb_model.predict(X_test)
print("Naive Bayes Results")
print("-----")
print("Accuracy:",accuracy_score(y_test, nb_pred))
print("Precision:",precision_score(y_test, nb_pred, average='weighted'))
print("Recall:",recall_score(y_test, nb_pred, average='weighted'))
```

```
print('Recall:', recall_score(y_test, nb_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, nb_pred, average='weighted'))
```

Naive Bayes Results

```
-----
Accuracy: 0.7332650273224044
Precision: 0.749562855390713
Recall: 0.7332650273224044
F1 Score: 0.6888259298377462
```

```
cm_nb = confusion_matrix(y_test, nb_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm_nb,annot=True,fmt='d',cmap='Blues',xticklabels=['negative', 'neutral', 'positive'],yticklabels=['negative', 'neutral', 'positive'],
plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

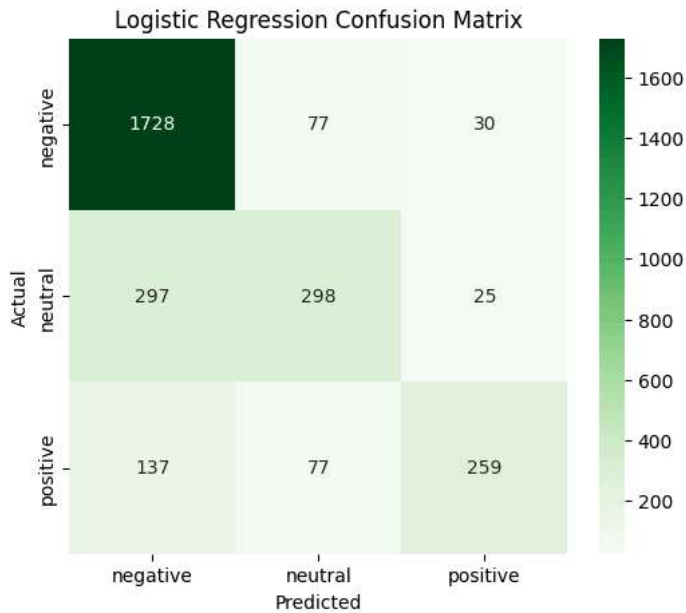


```
lr_model = LogisticRegression(
    max_iter=1000)
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
print("Logistic Regression Results")
print("-----")
print("Accuracy:", accuracy_score(y_test, lr_pred))
print("Precision:", precision_score(y_test, lr_pred, average='weighted'))
print("Recall:", recall_score(y_test, lr_pred, average='weighted'))
print("F1 Score:", f1_score(y_test, lr_pred, average='weighted'))
```

Logistic Regression Results

```
-----
Accuracy: 0.780396174863388
Precision: 0.7737543516480281
Recall: 0.780396174863388
F1 Score: 0.7659351166531798
```

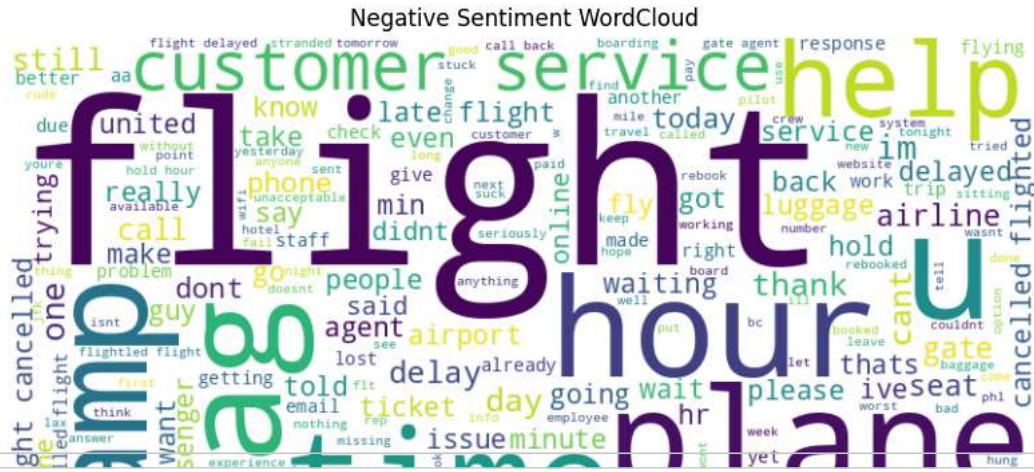
```
cm_lr = confusion_matrix(y_test, lr_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm_lr,annot=True,fmt='d',cmap='Greens',
    xticklabels=['negative', 'neutral', 'positive'],
    yticklabels=['negative', 'neutral', 'positive'])
plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



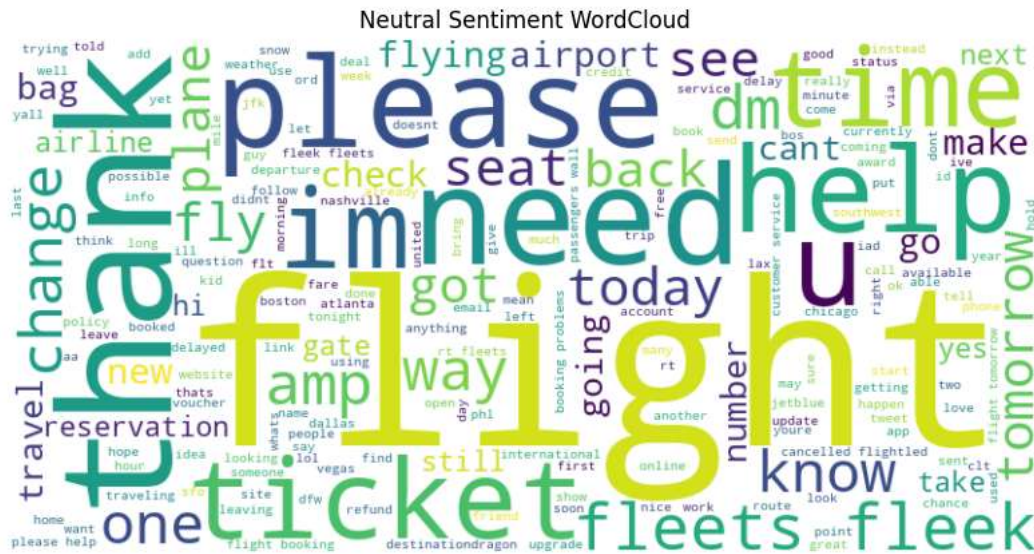
```
positive_text = ' '.join(df[df['airline_sentiment'] == 'positive']['clean_text'])
wordcloud_positive = WordCloud(width=800,height=400,background_color='white').generate(positive_text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud_positive, interpolation='bilinear')
plt.axis('off')
plt.title("Positive Sentiment WordCloud")
plt.show()
```



```
negative_text = ' '.join(
    df[df['airline_sentiment'] == 'negative']['clean_text']
)
wordcloud_negative = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(negative_text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud_negative, interpolation='bilinear')
plt.axis('off')
plt.title("Negative Sentiment WordCloud")
plt.show()
```



```
neutral_text = ' '.join(
    df[df['airline_sentiment'] == 'neutral']['clean_text']
)
wordcloud_neutral = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(neutral_text)
plt.figure(figsize=(10,5))
plt.imshow(wordcloud_neutral, interpolation='bilinear')
plt.axis('off')
plt.title("Neutral Sentiment WordCloud")
plt.show()
```



Start coding or generate with AI.